Chapter No 7 – Loop

C E – 101 : C & P F

Computing and Programming Fundamentals

Batch 2014

Sir Syed University of Engineering & Technology
Computer Engineering Department
University Road, Karachi-75300, PAKISTAN

Compiled By:

Syed Atir Iftikhar

CE - 101 : Computing and Programming Fundamentals

- **Course Objectives:**
  - Upon successful completion of this course, the student will be able to:
    - Fundamentals of Computer Engineering and Information Technology
      - Overview of History, Classification and Components of Computers
      - Number Systems and Logic Gates
      - Overview of Software, Operating Systems, Networks and Internet
    - Introduction about Programming
      - Basic Building Blocks, Loop, Decision Making

2014

CPF Books

- **Text Book:**

  1. **Introduction to Computers** (7th Edition)
     By Peter Norton
  2. **Turbo C Programming For The PC** (Revised Edition)
     By Robert Lafore

- **Reference Books:**

  1. **Computer, Communications and Information.**
     By Sarah Hutchinson and Stacey Sawyer
  2. **Let Us C**
     By Yashavant Kanetkar

Marks Distribution

Mid Term	_____	20
Lab Work + Project	_____	20
Quiz	_____	5
Assignment + Class Performance	_____	5
Semester Final Paper	_____	50
Total Marks	_____	**100**

- ***Performance Bonus*** _____ ***5 Marks***

$\mathcal{CPF}$

Course Instructors

- **Syed Aatir Iftikhar** satirs@hotmail.com

  Lecturer, CED

  Room No: BF-03

  Section B

2014

Course Outline

✓PART A

- History of Computer
- Classification of Computer
- Basic Components
- CPU, I/O, Peripheral Devices, Storage
- Von Neumann Architecture
- Number Systems
- Binary Numbers
- Boolean Logic
- Software
- Operating Systems
- Networks and Internet

✓PART B

- Types of Programming Languages
- Algorithm
- Flow Chart
- Introduction to C Language Programming
- Basic Building Blocks
- Loops
- Decision Making Statements

2014

$\mathcal{CPF}$

CE – 101: Computing and Programming Fundamentals

Chapter 7

Batch 2014

Loop

Compiled By: Syed Atir Iftikhar satirs@hotmail.com

Chapter 7 : Contents

2014

10

Background

- Almost always, if something is written worth doing, it is worth doing more than once. You can probably think of several examples of this form in real life, such as going to the TV program or eating a good dinner.
- Programming is the same; we frequently need to perform an action over, often with variations in detail each time.
- The mechanism that meets this need is the "**Loop**".

Types of Loop

- There are 3 types loop structures in C Language

1. For Loop
2. While Loop
3. Do-While Loop

For Loop

- It is often the case in programming that you want to do something a fixed number of times. Perhaps you want to calculate the paycheck for 100 employees or print out the squares of all the numbers from 1 to 50.
- The **for loop** is ideally suited for such cases.
- Let's look at a simple example of a for loop.

Program using For Loop

```
void main (void)
{
  int count;
  for ( count = 0; count < 10; count + + )
     printf ( " Count  =  %d \n ", count  );
  getch( );
}
```

Program using For Loop

Output:

```
Count = 0
Count = 1
Count = 2
Count = 3
Count = 4
Count = 5
Count = 6
Count = 7
Count = 8
Count = 9
```

- The printf ( ) function prints out the phrase " Count = " followed by the value of the variable count.

Structure of the For loop

- The parenthesis following the keyword "**for**" contains what we will call the "**loop**" expression.
- This loop expression is divided by semicolons into three separate expressions i.e the "**Initialize expression**", the "**Test expression**" and the "**Increment expression**".

for (count = 0; count < 10; count + +)

$\mathcal{C}\,\mathcal{P}\,\mathcal{F}$

Structure of the For loop

for (count = 0; count < 10; count + +)

Expression	Name	Purpose
Count = 0	Initialize Expression	Initialize loop variable
Count < 10	Test Expression	Test loop variable
Count + +	Increment Expression	Increment loop variable

Structure of the For loop

- **The Initialize Expression**
  - The Initialize expression, count = 0; initializes the count variable.
  - The Initialize expression is always executed as soon as the loop is entered.
  - We can start at any number, in this case we initialize the count to 0.
  - However, loops are often started at 1 or some other number.

Structure of the For loop

- **The Test Expression**
  - The second expression, count < 10, tests each time through the loop to see if count is less than 10.
  - To do this, it makes use of the relational operator for "less than" (<), If the test expression is true i.e count is less than 10, than the body of the loop i.e the printf ( ) statement will be executed.
  - If the expression becomes false i.e count is 10 or more, the loop will be terminated and control will pass to the statements following the loop.

Structure of the For loop

- **The Increment Expression**
  - The third expression, count + + increments the variable count each time the loop is executed.
  - To do this, it uses the increment operator (++).
  - The loop variable in a for loop does not have to be increased by 1 each time through the loop. It can be also decreased by 1.

$\mathcal{CPF}$

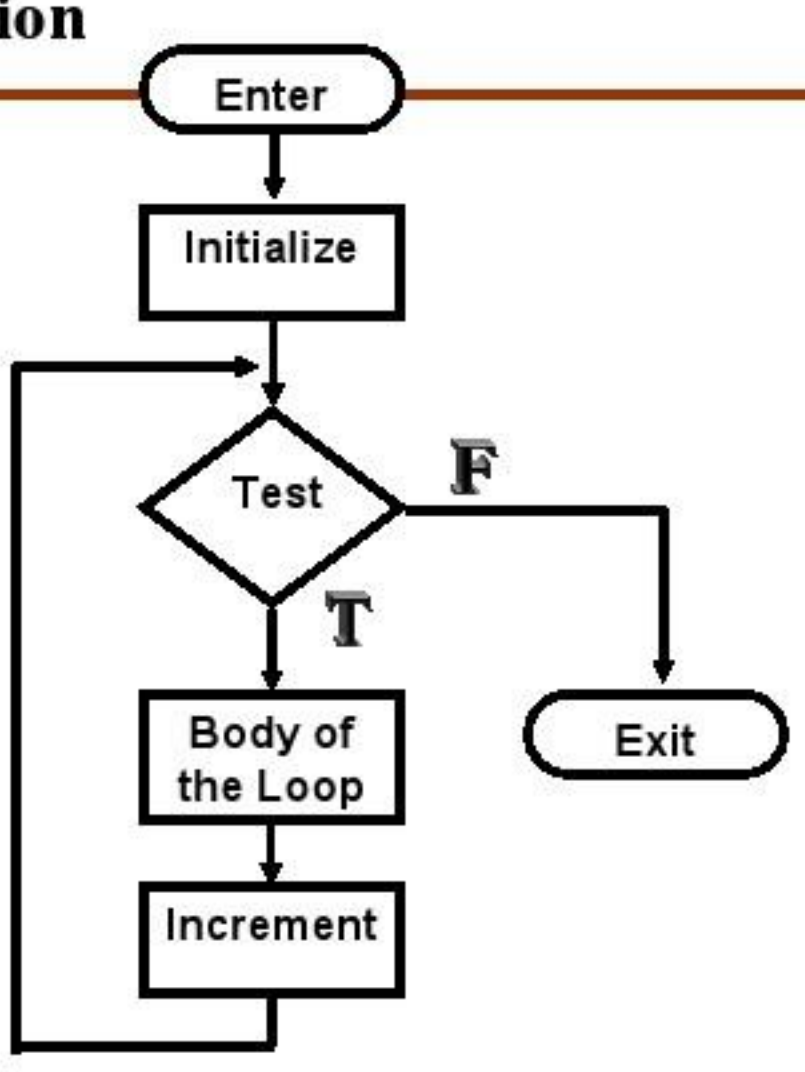
Structure of the For loop

- **The Body of the For Loop**
- Following the keyword for and the loop expression is the body of the loop: that is, the statement or statements that will be executed each time round the loop.

```
printf ( " Count  =  %d \n ", count );
```

- **Note**
- In a for loop, there is no semi colon between the loop expression and the body of the loop.

For Loop Operation



Multiple Statements in For Loop

```
void main (void)
 {
   int count , int total = 0;
   for ( count = 0; count < 10; count + + )
     {
       total = total + count;
       printf ( " Count = %d, Total = %d \n ",
                                    count, total );
     }
   getch( );
 }
```


Multiple Statements in For Loop

- **Output:**

```
Count = 0,  Total = 0
Count = 1,  Total = 1
Count = 2,  Total = 3
Count = 3,  Total = 6
Count = 4,  Total = 10
Count = 5,  Total = 15
Count = 6,  Total = 21
Count = 7,  Total = 28
Count = 8,  Total = 36
Count = 9,  Total = 45
```

- The most important new feature of this program is the use of the additional braces { { and } } to encapsulate the two statements that form the body of the loop.

Loop Style Note

- Many C programmers, handle braces in a some what different way than we have used in the above example:

```
for ( count = 0; count < 10; count + + )  {
      total  =  total  +  count;
      printf ( " Count  =  %d,  Total  =  %d \n ",
                                          count,  total );
}
```

- Both approaches are common but for the ease of the programmer and user, one should use one-brace per line approach.

Program

- **An ASCII Table Program**
  - In the PC family, each of the numbers from 0 to 255 represents a separate character.
  - From 0 to 31 are the control codes such as the carriage return, tab and linefeed and some of the graphics characters;
  - From 32 to 127 are the usual printing characters;
  - From 128 to 255 are graphics and foreign language characters.

Program

```
/* Print table of ASCII Character */
void main (void)
 {
   int n;   clrscr( );
   for ( n = 1; n < 256; n + + )
       printf ( " %3d  =  %c  \ t ", n, n );
   getch( );
 }
```


Program

- **Output:**

61= = 62 = > 63 = ? 64 = à 65 = A 66 = B

71= G 72 = H 73 = I 74 = J 75 = K 76 = L

- In C Language, we can use the same variable n, for both number and character; only the format specifier changes: %c prints the character, while %d prints the number. Format Specifier can interpret the same variable in different ways.
- The %d format specifier prints the decimal representation of n, while the %c format specifier prints the character whose ASCII codes is n.

Nested For Loop

- It is possible to nest one for loop inside another.

  For example a program that prints out the multiplication of table.

Program for multiplication of table

```
void main (void)
 {
  int cols, rows;
  for ( rows = 1; rows < 6; rows + + )  /* Outer Loop */
     {
       for ( cols = 1; cols < 6; cols + + ) /* Inner  Loop */
          printf ( " %3d  =  ", cols * rows );
     printf ( " \n " );
     }
   getch( );
}
```

Program for multiplication of table

- Output:

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25

- The inner loop steps through 5 columns, from 1 to 5, while the outer loop steps through 5 rows. For each row, the inner loop is cycled through once; then a new line is printed in preparation for the next row.
- Each time through the inner loop that is, at each intersection of a column and a row, the product of the row number (rows) and the column number (cols) is printed by the printf function.

Program for multiplication of table

- For instance, if the variable cols was 3, meaning we are on the third column, and rows was 4, meaning we are on the fourth row, then the program multiplies 3 by 4 and prints the product at the intersection of this row and column.
- Since we used the "less than" operator (<), the loop variable cols and rows never reach the limit of 6; i.e the loop both terminates at 5.

While Loop

- The second kind of the loop structure available in C Language is the "**While Loop**".
- Although at first glance this structure seems to be simpler than the for loop, it actually uses the same elements, but they are distributed throughout the program.
- The following program uses a while loop to reproduce the operation of our earlier for loop program, which prints the number from 0 to 9 and gave the running total.

Program using While Loop

```
void main (void)
 {
  int count , total = 0;
  while ( count  <  10 )
    {
      total  =  total  +  count;
      printf ( " Count  =  %d,  Total  =  %d \n ",
                  count + +, total );
    }
  getch( );
}
```

Program using While Loop

- **Output:**

```
Count = 0,  Total = 0
Count = 1,  Total = 1
Count = 2,  Total = 3
Count = 3,  Total = 6
Count = 4,  Total = 10
Count = 5,  Total = 15
Count = 6,  Total = 21
Count = 7,  Total = 28
Count = 8,  Total = 36
Count = 9,  Total = 45
```


Operation of the While Loop

- The loop variable count is initialize outside the loop in the declaration int count = 0.
- When the loop is first entered, the condition count < 10 is tested. If it false, the loop terminates. If it is true the body of the loop is executed.
- The increment expression is buried in the body of the while loop. When the printf ( ) statement that forms the loop body has finished printing, count is incremented by the (++) operator.

The Unexpected Conditions

- In situations, where the number if iterations in a loop are known in advance, as they are in the while loop example, while loop are actually less appropriate.
  - In this case the for loop is a more natural choice, since we can use its explicit initialize, test and increment expressions to control the loop.
- **So, when is the while loop becomes the appropriate choice?**
- The while loop shines, in situations where a loop may be terminated unexpectedly by conditions developing with in the loop.

Program using unexpected condition

```
void main (void)
{
  int count;
  clrscr( );
  printf ( " Type in a phrase:    \n  " );
  while ( getch ( ) ! = ' \r ' )
        count + + ;
        printf ( " \n Character Count is  =  %d ", count );
}
```

Program using unexpected condition

- **Output:**

  Type in a phrase: Pakistan
  Character Count is = 8

- This program invites you to type in a phrase. As you enter each character it keeps a count of how many characters you have typed, and when you hit the [Return] key it prints out the total.
- **Note**
  - **" While Loop are more appropriate than the For Loop, when the condition that terminates the loop occurs unexpectedly."**

Why While Loop

- The loop on this program terminates when the character typed at the keyboard is the [Return] character.
- There is no need for a loop variable, since we do not have to keep track of where we are in the loop, and thus no need for initialize or increment expressions.
- Since there is no loop variable to initialize or increment, thus the while loop, consisting only of the test expression, is the appropriate choice.

Program

```
void main (void)
 {
   int count  =  -1;       char ch    =  'a';
   printf ( " Type in a phrase:    \n  " );
   while ( ch  ! = ' \r ' )
        {
                ch  =  getche ( );
                count + + ;
        }
   printf ( " \n Character Count is  =  %d ", count );
}
```


Program

- **Output:**

```
Type in a phrase:       Pakistan
Character Count is  =  8
```

- In this program, count variable must be initialized to an odd looking –1, value because the program now checks with character is read after the loop is entered instead of before.

ASCII Revised Program

```
void main (void)
 {
   char ch  =  'a';
  while ( ch  ! = ' \r ' )
   {
     printf ( " Enter a Character:  \n  " );
     ch  =  getche ( );
     printf ( " \n The code for %c is %d.  \n  ", ch, ch  );
   }
 }
```


ASCII Revised Program

Output:

```
Enter a Character:
a
The code for a is 97
Enter a Character:
b
The code for b is 98
Enter a Character:
A
The code for A is 65
```

Nested While Loops

```
void main (void)
 {
   int m;  char  ch  =  'a';
   for ( m  =  0; m  <  5; m + + )
    {
        printf ( " \n Type in a letter from 'a'  to  'e'  : \n " );
        while ( ( ch  =  getche ( ) )  !=  'd' )
     {
        printf ( " \n Sorry, %c is incorrect. \n ", ch  );
        printf ( " Try again. \n " );
      }
          printf ( " \n That's It ! \n " );
    }
   printf ( " Game's Over ! \n " );
}
```


Nested While Loops

- This program let you guess a lower case letter 'a' to 'e' and tells you if you are right or wrong.
- **Output**

```
Type in a letter from 'a' to 'e' :
a
Sorry, a is incorrect.
Try Again
c
Sorry, c is incorrect.
Try Again
d
That's It !
```

Miscellaneous Stuff

- Assignment Expression as values in a loop

```
while ( ( ch = getche ( ) ) != ' d ' )
ch = getche ( )
ch = 'a'
'a' != 'd'
```


Miscellaneous Stuff

- Precedence: Assignment versus relational operators.

```
while ( ( ch = getche ( ) ) != ' d ' )
```

- If the parentheses were not there, the compiler would interpret the expression like this:

```
while ( ch = ( getche ( ) != ' d ' ) )
```

- This of course is not what we want at all, since ch will now be set equal to the results of a true/false expression. The reason we need the parenthesis is that the precedence of the relational operator (!=) is greater than of the assignment operator (=).
- So unless parentheses, tell the compiler otherwise, the relational operator (! =) will be evaluated first.

Mathematical WHILE loop

```
void main (void)
 {
   long answer , number = 1;
   while ( number !=  0  )
     {
        printf ( " \n Enter a number: ");
        scanf ( " %ld ", &number );
        answer = 1;
        while ( number  >  1 )
           answer = answer * number - - ;
      printf ( " Factorial is %ld \n ", answer );
      }
 }
```


Mathematical WHILE loop

- Output

```
Type number:      3
Factorial is:     6
Type number:      4
Factorial is:     24
Type number:      0
```

- This program uses outer while loop to recycle until a 0 value is entered. The inner loop uses the document operator to reduce the variable number-- which starts out at the value typed in number reaches to 1, the loop terminates.

Do-While Loop

- The last of the three loops in C Language is the **do-while Loop**.
- This is very similar to the while loop except that the test occurs at the end of the loop body.
- This guarantees that the loop is executed at least once before continuing.

Do-While Loop

- Such a setup is frequently used where data is to be read. The test then verifies the data, and loops back to read again if it was unacceptable.

```
do
  {
    printf (" Enter the number :  ");
    scanf ("%d", &num);
  }
while ( num < 25 )
```

Do-While Loop

```
void main (void)
 {
   int count = 0 , int total = 0;
   do
      {
        total = total + count;
        printf ( " Count = %d, Total = %d \n ",
                                        count + +, total );
      }
   while ( count < 10 ) ;
   getch( );
}
```


Do-While Loop

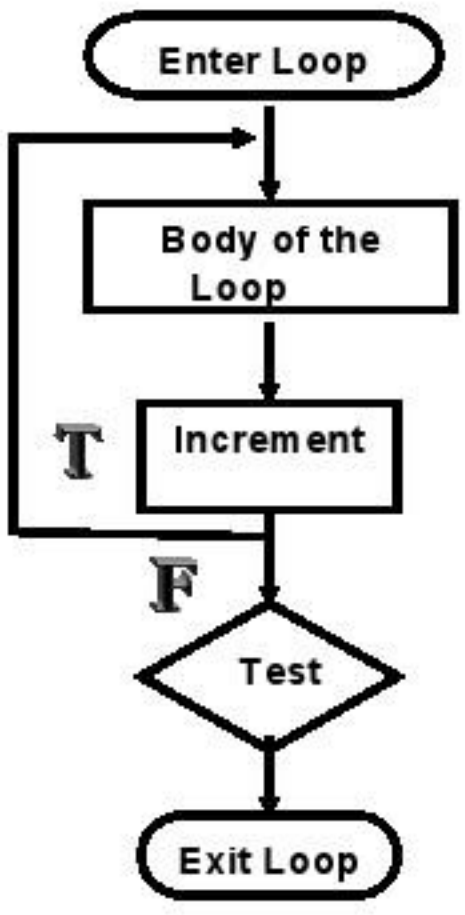
- **Output:**

```
Count = 0,  Total = 0
Count = 1,  Total = 1
Count = 2,  Total = 3
Count = 3,  Total = 6
Count = 4,  Total = 10
Count = 5,  Total = 15
Count = 6,  Total = 21
Count = 7,  Total = 28
Count = 8,  Total = 36
Count = 9,  Total = 45
```

Do-While Loop

- The operation of the Do while Loop is sort of an upside-down version of the While loop.
- The body of the loop is first executed, then the test condition is checked.
- If the test condition is true, the loop is repeated; if it is false, the loop terminates.
- The body of the loop will always be executed at least once, sine the test condition is not checked until the end of the loop.
- The do keyword marks the beginning of the loop, it has no other function.
- The while keyword marks the end of the loop and contains the loop expression.

Do-While Loop Operation



Summary of Loop in C Language

- The **For Loop** is frequently used, usually where the loop will be traversed a fixed number of times. It is very flexible, and novice programmers should take care not to abuse the power it offers.
- The **While Loop** keeps repeating an action until an associated test returns false. This is useful where the programmer does not know in advance how many times the loop will be traversed.
- The **Do While Loop** is similar to the While Loop, but the test occurs after the loop body is executed. This ensures that the loop body is run at least once.

Class Assignment No 3

1. Differentiate between the different types of loop available in C Language with some suitable examples.
2. Write a program that's continuously takes the input from the user and calculates the factorial of the input number.
3. Differentiate between the While loop and Do-While loop with regards to the flow chart.